

C++ Review

The goal of this lecture is to go over what you should have learned in the prerequisite C++ Programming course, CSE233, that you should have taken before this. This is not an exhaustive explanation of every topic. I'm just providing an overview to remind you of what you should have learned and are expected to know coming into this class. All the following lectures will assume you know this stuff. If you see something here you don't remember, you should go back to your material and book from the previous course and remind yourself of it before moving on with this course.

Basic Program

Let's look at the very basic C++ "Hello World" program below:

```
#include <iostream>

int main() {
    std::cout << "Hello World!" << std::endl;
    return 0;
}
```

At the top, we have this "include" directive: `#include <iostream>`. This tells the compiler to find a file called "iostream" and include it with the source code in the file we are compiling. Since it is wrapped in angle brackets, the less-than and greater-than symbols, the compiler expects this to be a C++ standard library header and automatically searches for it there. In a couple lectures we'll see how to include a local file.

Moving on, we have the function `int main()`. This is the entry point for every C++ program. The compiler searches for a function called main

when it creates the final executable to know where to start the program. So, you can't have an executable program without an `int main()`, and you also can't have multiple `main` functions in files you are linking together into an executable. And, of course, you should remember that the curly braces "{}" delineate the start and end of the function.

The first statement in the function is `std::cout << "Hello World!" << std::endl;`. In your past class, you may have had a `using namespace std;` somewhere in your program, but `using namespaces` is generally discouraged, so those are banned in this class. You will lose points if you use them. Instead, we are prefixing the `cout` object with its namespace, `std`. The two colons that go between them are called the scope-resolution operator.

After `std::cout`, we have the insertion operator, which is two "less than" symbols. There can't be any spaces between the two symbols. After that, we have a string literal, "Hello World!". We are chaining our print statement here by placing another insertion operator after the string literal, and adding `std::endl` to the output. This prints a new line.

Finally, we have `return 0;`. This is the code that is returned to the operating system that tells it the program exited normally. And note that in C++ all statements have to end with a semicolon.

Variable Declaration Statements

Variables allow us to associate names with memory locations, that way we can use a value by name throughout our program. To use a variable, we first need to declare it. To declare a variable, we need to provide a data type, a name, and, optionally, an initialization. Below are declarations, one without an initialization, and the other with.

```
int x;
```

```
int y = 0;
```

Both these variables have the data type `int`. The first one is called "x", and the second one is called "y". Only y is being initialized here, which was done by the "= 0".

In C++, if we don't initialize a variable, it could have a garbage value. There's no guarantee in the C++ standard that the compiler will assign a default value, so it is best to assume that it will read whatever data is currently in memory at the location that is allocated for that variable. If a variable is not initialized, we need to make sure it is given a value, either through a later assignment or reading input, before we try using its value.

Constants

We may want to declare variables whose values are never changed during the lifetime of our program. We can do so using the `const` keyword in front of the data type of the variable:

```
const double pi = 3.1415926;  
const double e = 2.78128182;
```

Arrays

We can also declare arrays, which are collections of elements of the same type. You use square brackets at the end of a variable name to denote that it is an array (note that these are static arrays, we'll discuss dynamic arrays at the end of the lecture). Arrays declared like this must be given a size that is known at compile-time.

```
int arr[5];
```

Since 5 is an integer literal, it is known at compile-time and can be used.

Flow Control

C++ offers a few different methods for flow control, which I'll briefly touch on here.

If Statements

The simplest flow control is an `if` statement. You can have a bare `if`, or an `if` followed by an `else`. You can avoid nesting by creating `else if` statements.

```
int main() {
    int num;

    std::cout << "Enter a number: ";
    std::cin >> num;

    if (num > 0) {
        std::cout << "Your number is positive" << std::endl;
    }
    else if (num < 0) {
        std::cout << "Your number is negative" << std::endl;
    }
    else {
        std::cout << "Your number is zero" << std::endl;
    }

    return 0;
}
```

While

A while loop executes as long as a given condition is true. They are most useful when the number of times the loop must execute is not known at compile-time. Below shows an example of using a while loop:

```
int main() {
    int input;
    int total = 0;
    char response = 'y';
    while (tolower(response) != 'n') {
```

```

        std::cout << "Enter a value: ";
        std::cin >> input;
        total += input;
        std::cout << "Continue? (y/n): ";
        std::cin >> response;
    }
    std::cout << "Total = " << total << std::endl;
    return 0;
}

```

For

A for loop can and should be used in almost all cases when the number of iterations is known at compile time.

```

int main() {
    std::vector<int> values = { 1, 2, 3, 4, 5 };
    for (int i = 0; i < values.size(); ++i) {
        std::cout << values[i] << std::endl;
    }
    return 0;
}

```

Switch

A switch statement can be used in place of a chain of `else if` statements when the values being compared are an integer-based type (any variety of `int`, `char`, or `enum`). You write a `case` statement for each expected value. You should always write a `default` case in the event that an invalid control value is entered. Note that you do not need curly braces around the individual cases. A `break;` statement needs to be added at the end of each case to avoid falling through to the `default` case.

```

int main() {
    int floor = 0;
    std::cout << "You are currently on the ground floor" << std::endl;
    std::cout << "Enter the floor you'd like to change to (1-5): ";
    std::cin >> floor;
}

```

```

switch (floor) {
    case 0:
        std::cout << "You are still on the ground floor" << std::e
        break;
    case 1:
        std::cout << "You are on the first floor" << std::endl;
        break;
    case 2:
        std::cout << "You are on the second floor" << std::endl;
        break;
    case 3:
        std::cout << "You are on the third floor" << std::endl;
        break;
    case 4:
        std::cout << "You are on the fourth floor" << std::endl;
        break;
    case 5:
        std::cout << "You are on the fifth floor" << std::endl;
        break;
    default:
        std::cout << floor << " is an invalid selection" << std::e
}
return 0;
}

```

Functions

Functions help us modularize and reuse code. We can declare and define functions. To declare a function, we need to provide a return type, name, parameter list, and we end with a semicolon. The return type, name, and parameter list are called the function's signature. To define a function, we provide the signature, but use curly braces in place of the semicolon. We can then write whatever statements the function needs to execute between the curly braces. Below is an example of a function declaration, and a definition for the function we declared.

```

// declaration
int add(int x, int y);

// definition

```

```
int add(int x, int y) {  
    return x + y;  
}
```

The function is called `add` and returns an integer. It has two parameters, which are both integers. The declaration ends with a semicolon, and has to come before the definition. The definition has curly braces, and contains a single `return` statement. The value returned is the value of the two parameters added.

void Functions

A function that returns `void` is one that doesn't return a value. They can still contain `return` statements, but there can't be an expression after the keyword `return`.

```
void print(int x) {  
    std::cout << x << std::endl;  
    // return is optional  
    return;  
}
```

The function above prints the value of `x`, but does not return any values.

Pass-by-Reference Parameters

The previous functions have all used pass-by-value parameters. Pass-by-value creates a copy of the value passed as an argument when the function is called, and stores it in a new variable associated with the function. Any changes to the parameter in the function do not impact the argument outside of it.

```
void triple(int x) {  
    x *= 3;  
}  
  
int main() {  
    int value = 4;  
    triple(value);  
}
```

```
// the following statement will print 4
std::cout << value << std::endl;
return 0;
}
```

In the example above, we have this function `triple` that takes an integer as a pass-by-value parameter. Even though we multiply the parameter by 3 in the function, after the function is called in `main`, the variable `value` will still be 4. If we use a pass-by-reference parameter, we can make it so the change inside the function is reflected outside of it:

```
void triple(int& x) {
    x *= 3;
}
int main() {
    int value = 4;
    triple(value);
    // now this statement will print 12
    std::cout << value << std::endl;
    return 0;
}
```

To make a parameter pass-by-reference, we add an ampersand (and symbol) to the end of the parameter's type.

Pass-by-reference passes the memory address to the function instead of the variable's value. Rather than creating a new variable that is associated with the function, the parameter and argument are now accessing the same place in memory.

If we want to provide a guarantee that the value will not be changed by the function, but still want to pass it by reference, we can use the

`const` keyword to specify that the parameter should be immutable.

This is useful for when we are passing object that potentially use a large amount of memory (making them expensive to copy), but we want to guarantee the function will not modify the parameter.

```
void print(const std::string str) {
```

```
std::cout << str << std::endl;
// can't do this, str is const:
// str += "!!!";
}
```

Arrays as Parameters

We can pass an array as a parameter to a function by including empty array brackets at the end of the parameter's name. See below:

```
void readArray(int arr[], int size) {
    for (int i = 0; i < size; ++i) {
        std::cin >> arr[i];
    }
}
```

Behind the scenes, passing an array is the same as passing a pointer (which we'll discuss at the end of this lecture) to the first element of the array. Since a pointer is a reference to a memory address, array parameters are implicitly pass-by-reference.

Like with other pass-by-reference parameters, we can use the `const` keyword to specify that the array should not be modified by the function:

```
void printArray(const int arr[], int size) {
    bool printComma = false;

    for (int i = 0; i < size; ++i) {
        if (printComma) std::cout << ", ";
        std::cout << arr[i];
        printComma = true;
    }
    std::cout << std::endl;
}
```

Structs and Classes

Structs and classes allow you to implement more complex, user-defined

data types. Typically, structs are used to implement data types that are purely data or have very limited functionality, while classes are used to implement data types that have more complex functionality associated with them. This is stylistic and idiom based, as far as the compiler is concerned the only difference between structs and classes are that structs have public access and inheritance by default, and classes have private access and inheritance by default.

Due to them being functionally the same, we will use classes for all the examples.

To declare a class (or struct), write the keyword `class` (or `struct`, respectively), followed by the name of the class. The name can be followed by a semicolon if you are forward declaring a class (we will discuss in a future lecture why you'd want to do this), or curly braces to declare/define the class. Inside the curly braces, we put our fields (member variables) and methods (member functions). C++ uses access regions, specified by `public:`, `protected:`, and `private:`. This is different to some other object-oriented languages, like Java or C#, that place a `public`, `protected`, or `private` specifier on each member. Below are a few basic examples:

```
// forward declaration
class Engine;

// defined class
class Wheel {
public:
    Wheel(double wheelDiameter) {
        diameter = wheelDiameter;
    }
    double getDiameter() const {
        return diameter;
    }
    double getSpeed(int rpm) const {
        double circumference = 3.14 * diameter;
        int rph = rpm * 60;
        return (rph * circumference) / 63360;
    }
private:
```

```

    double diameter;
}; // note that you need a semicolon after the closing curly brace for

// declared class
class Car {
public:
    Car(std::string carMake, std::string carModel, Engine carEngine, W
    double getCurrentSpeed() const;
    // ...
private:
    std::string make;
    std::string model;
    Engine engine;
    Wheel wheel;
};

```

The class `Engine` has been forward declared. We have used the type `Engine` for a field in the class `Car`, but we haven't used any of its members. The class `Wheel` is fully defined, since we provided definitions for all the methods inside the class. The class `Car` has been declared, since we've provided all the field and function declarations, but haven't defined any of them.

Note that some of the methods have a trailing `const` specifier. This means that the method will not modify any of an object's fields when the method is called.

If we wanted to define the methods for the class `Car` outside of the class, we would need to put `Car::` in front of the method's name.

```

Car::Car(std::string carMake, std::string carModel, Engine carEngine,
    make = carMake;
    model = carModel;
    engine = carEngine;
    wheel = carWheel;
}

double Car::getCurrentSpeed() {
    // ...
}

```

Pointers and Dynamic Arrays

A pointer is a variable that stores a memory address instead of a value. A dynamic array is an array that uses a pointer to reference a sequence of memory addresses. This allows for memory for an array to be allocated at runtime, bypassing the limitation of static arrays where their size must be known at compile time.

To declare a pointer, an asterisk should be placed before the variable's name. To create a raw pointer that creates new storage, use the `new` keyword. Alternatively, a pointer can be assigned to an existing variable's memory address. If the `new` keyword was used to allocated memory, it must be freed using `delete`.

```
int main() {
    int x = 5;
    // use & to access the memory address of x
    int *ptr = &x;

    // create a raw pointer
    int *rawPtr = new int;
    // access data that pointer is referencing
    // by dereferencing pointer with *
    *rawPtr = 16;

    int size;
    std::cout << "Enter size of array: ";
    std::cin >> size;
    // declare a dynamic array with size determined at runtime
    int *arr = new int[]
    // the dynamic array is used the same way a static array would be
    for (int i = 0; i < size; ++i) {
        std::cout << "Enter a number: ";
        std::cin >> arr[i];
    }

    // we need to delete rawPtr
    delete rawPtr;
    // we also need to delete arr
    delete[] arr;
    // we do not delete ptr, however, since it is referencing a normal
    return 0;
```

```
}
```

Pointers as Parameters

Pointers can be used as function parameters. They operate similar to pass-by-reference, in that changes to the value the pointer is referencing will be reflected outside the function. However, unlike pass-by-reference, pointer parameters must be explicitly dereferenced to access the value they reference, whereas pass-by-reference parameters can be used like normal variables. It is also possible to create memory leaks or undefined behavior by changing a pointer reference. Below are two examples of a swap function, one using pass-by-reference and the other using pointers:

```
void swapByRef(int& a, int& b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}  
void swapByPtr(int *a, int *b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}  
// you could also swap the pointers  
void swapPointers(int *a, int *b) {  
    int *temp = a;  
    a = b;  
    b = temp;  
}
```

Summary

As stated at the beginning, the point of this lecture isn't to teach you every nuance from an intro class in C++. You should have already taken that. But, if anything presented here seems unfamiliar, this gives you a chance to go back and review it now before we get too deep into the class.

